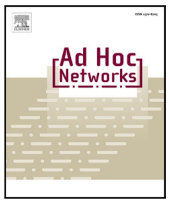
Ad Hoc Networks 176 (2025) 103887



Contents lists available at ScienceDirect

Ad Hoc Networks

journal homepage: www.elsevier.com/locate/adhoc





ESFMTO: A reliable task offloading strategy based on edge server failure model in IIoT

Yubin Yang [a], Yan Chen [a], Ningjiang Chen [a,b,c,*], Juan Chen [a]

[a] School of Computer and Electronic Information, Guangxi University, Nanning, 530004, China
[b] Key Laboratory of Parallel, Distributed and Intelligent Computing (Guangxi University), Education Department of Guangxi Zhuang Autonomous Region, Nanning, 530004, China
[c] Guangxi Center of Technology Innovation for Intelligent Digital Services, Nanning, 530004, China

ARTICLE INFO

Keywords:
Industrial internet of things
Edge server failure
Bayesian neural network
Deep reinforcement learning

ABSTRACT

The extensive use of automation equipment and sensors in the Industrial Internet of Things (IIoT) has led to a significant increase in data volume, which has placed higher demands on the real-time processing capability. Edge computing enables real-time response and rapid decision-making by offloading data processing to the edge of the network. However, the complexity of the industrial production environment leads to edge server failures, which seriously affects the system stability and security. To address this issue, this paper develops an edge server failure model for IIoT, analyzing the interaction between the hardware failure and the container failure during the failure occurrence and recovery. Further, based on the edge server failure model, a task offloading strategy named ESFMTO is proposed, which employs the SAC-BNN (Soft Actor-Critic with Bayesian Neural Network) algorithm. The probability distribution of the task completion time is updated through the Bayesian Neural Network (BNN), accurately evaluating the Conditional Value at Risk (CVaR). During the training process of Deep Reinforcement Learning (DRL), a perturbation neural network is introduced to perturb the input state, which enhances the system robustness under uncertain failure conditions. Unlike existing approaches that often assume each edge server hosts a single container, the paper explicitly considers multi-container deployment in IIoT, bridging the gap between theoretical assumptions and real-world industrial requirements. Experimental results demonstrate that SAC-BNN outperforms existing methods in dealing with the edge server failure, reducing CVaR by at least around 0.9 and improving task completion rates by at least 13.86% compared to the baseline algorithms.

1. Introduction

The Industrial Internet of Things (IIoT) combines cloud computing, big data, and artificial intelligence, significantly optimizing manufacturing processes and improving product quality. With the widespread use of automated devices and sensors, systems need to handle large amounts of data and make decisions within an extremely short time to avoid losses caused by production line stagnation or equipment failure [1]. To address the challenges, Edge Computing (EC) has emerged, which provides low-latency, high-bandwidth, and high-reliability services for IIoT by deploying computational and storage resources at the edge close to users [2]. Task offloading is a critical challenge in EC. Offloading computation-intensive tasks from terminal devices to edge servers can not only effectively reduce delay and alleviate device load, but also significantly improve the real-time response and Quality of Service (QoS).

Although task offloading has significantly improved the computational efficiency of EC, edge server failure is still a non-negligible problem [3]. In scenarios such as automobile manufacturing where timeliness is extremely important, edge server failures may prevent critical tasks from being completed on time, leading to production line disruptions, equipment damage, even safety accidents and significant economic losses. Existing studies [4–6] have explored the impact of edge server failures on task offloading. However, these studies typically consider single failure or assume that each server runs only one container, which is difficult to reflect the complexity of actual industrial production. In actual production environments, each edge server usually runs multiple containers simultaneously to support multi-task and resource sharing. The simplified assumptions limit their applicability in real industrial manufacturing scenarios. In addition, most existing task offloading methods aim to minimize latency, but struggle to deal

* Correspondence to: School of Computer and Electronic Information, Guangxi University, Nanning, 530004, China.
E-mail address: chnj@gxu.edu.cn (N. Chen).

https://doi.org/10.1016/j.adhoc.2025.103887
Received 19 January 2025; Received in revised form 8 April 2025; Accepted 19 April 2025
Available online 4 May 2025
1570-8705/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

with fluctuations and extreme situations of task latency in complex environments. Therefore, besides optimizing the average latency, it is also important to focus on the risks of extreme latency, ensuring that the system remains stable under various contingencies and meets the high reliability requirements of industrial manufacturing.

In the paper, an edge server failure model for IIoT is designed, with task completion rate and delay risk as dual optimization objectives. Based on the failure model, a task offloading strategy is proposed, named ESFMTO. The strategy dynamically optimizes task offloading decisions through Deep Reinforcement Learning (DRL) and incorporates Conditional Value at Risk (CVaR) into the DRL reward function through Bayesian Neural Networks (BNN). The main contributions of the paper are as follows:

- A realistic edge server failure model is proposed, assuming that each server runs multiple containers simultaneously. It deeply investigates the correlation between the edge server hardware failure and the container failure and analyzes the interaction mechanism between these two types of failures during occurrence and recovery.
- A task offloading strategy ESFMTO based on the edge server failure model is proposed and utilizes the SAC-BNN (Soft Actor-Critic with Bayesian Neural Network) algorithm for offloading decisions. BNN is adopted to update the distribution of task completion time and calculate CVaR in real-time. The CVaR is incorporated into the reward function of Soft Actor-Critic (SAC) to reduce the potential risk associated with the edge server failure.
- A perturbation generation network is introduced to improve the reliability of the SAC-BNN algorithm in uncertain environments. The network applies perturbations to the input state to generate the perturbed state, which is then fed into the actor network and critic network for gradient update.

The rest of the paper is organized as follows. Section 2 introduces related work. Section 3 gives the model and the problem description. The proposed approaches are described in Section 4. Section 5 conducts the experiments and analyzes the results. Finally, conclusions and future work are given in Section 6.

2. Related work

To optimize task offloading decisions and reduce latency, researchers have proposed a variety of methods, such as delay-optimized task offloading, failure-based task offloading, and DRL-based task offloading. The Section summarizes the status of these methods and explores the challenges they face in practical applications.

(a) Delay-based task offloading

Due to the complexity of tasks in IIoT, effectively reducing the delay of task offloading has become a significant challenge. Numerous approaches have been proposed to reduce the delay of task offloading and improve the overall system performance. Zhu et al. [7] minimized computational latency by jointly optimizing the NOMA-based transmission time and workload offloading allocation. Bai et al. [8] proposed an online collaborative task offloading algorithm based on Lyapunov optimization to reduce the system delay within a specified energy budget. Binh et al. [9] proposed an advantage-oriented task offloading algorithm with dueling actor-insulator network to minimize the task timeout. Ai et al. [10] combined Reinforcement Learning (RL) and DRL to predict the channel state, analyzing network conditions and task characteristics in real time. To minimize vehicular task latency under the long-term energy constraints, Dai et al. [11] decoupled these constraints by Lyapunov optimization and used the Markov approximation to dynamically adjust the offloading strategy. Binh et al. [12] combined fuzzy logic and Deep Q Network (DQN) to tackle the delay-constrained task offloading in the vehicular edge computing network, significantly improving the initial performance and overall stability of task offloading.

(b) Failure-based task offloading

While addressing the challenge of the delay, researchers have also focused on the failure caused by uncertainties, such as network conditions, server state, and communication channel, which can affect the QoS of the system. Several studies have incorporated failures into task offloading strategies to improve the system robustness and reliability. EvWOA [4] employed the Poisson diffusion process to analyze server and communication connection failures. Long et al. [5] proposed a semi-online fault-tolerant offloading strategy to address the MEC failure, the strategy employed the adversarial Dueling Deep Q-Network algorithm to determine user offloading behavior. Liang et al. [6] quantified the impact of server failure and recovery rates to assess edge server reliability. To solve the reliability issue caused by the uncertainty of communication channels, ROMA [13] combined Lyapunov optimization to transform long-term reliability constraints into instantaneous objectives. Tang et al. [14] incorporated task failures, network failures, and computing resource failures into the objective function and constraints, and applied Lyapunov optimization to decompose the complex problem into per-slot subproblems. In response to the computing resource failure caused by cyberattacks and hardware failures, Qu et al. [15] employed the Fast-CRO algorithm to jointly optimize total task execution time and delay of critical tasks. Considering the failure and recovery of edge servers, Liang et al. [16] modeled the server state transition process as a Markov model and used a Dueling Double Deep Q Network (D3QN)-based approach to optimize the discrete action space of task offloading. However, few studies have explored edge server failure models in detail, particularly the complex interactions and dynamic transitions between edge servers and multiple containers they host.

(c) DRL-based task offloading

Traditional task offloading methods often struggle to cope with the dynamic and high-dimensional challenges in IIoT. DRL, with its powerful decision-making and optimization capability, has become an important approach to solve task offloading problems. ISAC-CPTORA [17] incorporated distributional RL and prioritized experience replay to optimize task offloading and resource allocation for IIoT devices. PA-MADRL [18] constructed noise based on a differential privacy mechanism and added the noise to the process of action selection and policy update, preventing offload preference inference attacks. Zhao et al. [19] adopted the Deep Deterministic Policy Gradient (DDPG) algorithm for task offloading and scheduling optimization in the high-dimensional continuous action space. DODQ [20] modeled mobile applications as directed acyclic graphs and employed DQN to train the decision model, adapting to the dynamic environment and generating task offloading schemes flexibly. MaDRLAM [21] combined the Transformer and Pointer networks to optimize the task priority setting and the computational node selection, utilizing a multi-actor and single critic strategy for training. Pamuklu et al. [22] proposed a task offloading method based on multi-agent risk-sensitive reinforcement learning. The method dynamically adjusts energy and deadline constraints, effectively reducing the violation rate and extending the hovering time.

The above studies have provided various approaches for task offloading. However, some work has ignored the impact of different failures on system performance, or have only taken average delay as the optimization target. Furthermore, the impact of the edge server failure on system reliability is rarely explored in depth. Therefore, this paper focuses on the edge server failure and proposes a task offloading strategy based on the edge server failure model.

3. System modeling and problem description

3.1. Scenario

To solve the delay of task offloading in IIoT, an automobile manufacturing plant is used as the application scenario, as shown in Fig. 1.

Table 1
Variables and definitions.

Variables	Definitions	Variables	Definitions
u_i^t	Task generated by user i at time step t	A_C	Container failure state
L_{th}	Communication link throughput	θ	Hardware failure rate of the edge server
$T_{tr}(u_i^t)$	Transmission delay of task u_i^t	λ	Container failure rate
ζ_i^t	Computation delay to successfully complete task u_i^t	μ	Container recovery rate
c_i^t	The number of CPU cycles required for task u_i^t	η	Hardware recovery rate
f_i^t	Computational resources allocated to u_i^t	$q_{i,t}^{(k,A_H)}$	Probability that task u_i^t fails due to hardware failure
$F_{i,t}^{(k,A_H)}$	Hardware failure time of server e_j during the k-th execution	$q_{i,t}^{(k,A_C)}$	Probability that task u_i^t fails due to container failure
$R_{i,t}^{(k,A_H)}$	Recovery time of server e_j during the k-th execution	$q_{i,t}^k$	Failure probability of task u_i^t during the k-th execution
$F_{i,t}^{(k,A_C)}$	Failure time of container e_j^v during the k-th execution	$P_{i,t}^{(k,A,B)}$	Transition probability from state A to state B
$R_{i,t}^{(k,A_C)}$	Recovery time of container e_j^v during the k-th execution	$T_{i,t}^n$	Time required to resume after k-th failure
φ_k	State subset for the k-th execution	$F_{i,t}^k(x)$	The cumulative distribution function of $T_{i,t}^n$
A_k	The k-th execution state	$p_{i,t}(N)$	Success probability on the N-th execution
A_{k+1}	Transition to the $(k+1)$-th execution	L_i^t	Total computation delay of task u_i^t
A_H	Hardware failure state	$T_q(u_i^t)$	Queuing delay of task u_i^t

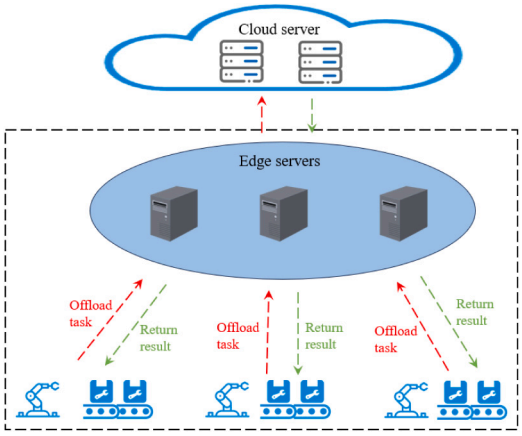


Fig. 1. Task offloading in the automotive manufacturing plant environment.

There are multiple edge servers, each of which is configured with multiple containers and deployed close to the production line to ensure high-speed and low-latency data transmission. Each production line is equipped with testing equipment for regular quality inspections on automotive components or complete vehicles. Quality inspections generate a large amount of real-time data, including high-resolution images, physical parameters such as temperature and pressure collected by sensors, and information about equipment operating status. The real-time data is offloaded to the edge server for processing, then the processed results, including qualified assessments, production parameter adjustments and equipment maintenance recommendations, are immediately returned to the industrial equipment. The industrial equipment will immediately remove defective products and adjust production parameters according to the results, ensuring product quality and production efficiency.

3.2. Problem description

The task offloading process is modeled as follows. The total system operating time is divided into a sequence of consecutive time slots, denoted as $\{1, 2, \ldots, t_{\max}\}$, each with a duration of τ. There are m edge servers deployed in the factory, represented as $\{e_1, e_2, \ldots, e_m\}$. Each edge server is configured with c containers, and is represented as $\{e_j^1, e_j^2, \ldots, e_j^c\}$, where each container executes tasks independently. There are n industrial application users in the factory, denoted as $\{u_1, u_2, \ldots, u_n\}$. At the beginning of each time slot $t \in \{1, 2, \ldots, t_{\max}\}$, each user generates a task. $\{u_1^t, u_2^t, \ldots, u_n^t\}$ represents the task set at time slot t, and d_i^t denotes the task size of task u_i^t. w_i^t denotes the offloading decision for task u_i^t, indicating that task u_i^t is offloaded to edge server w_i^t for processing. The primary variables and their definitions are summarized in Table 1.

Task offloading delay comprises three components: transmission delay, queuing delay, and computation delay. These three types of delay are described in detail as follows.

A. Task Transmission Delay

In wireless communication systems, Shannon's theorem [23] proves that under a given channel bandwidth and signal-to-noise ratio (SNR), there is an upper bound on the throughput achievable for reliable communication. In the paper, Shannon's theorem is employed to compute the communication link throughput between the user and the edge server. Accordingly, the communication link throughput L_{th} is expressed as follows:

$$L_{th} = b \log_2 \left(1 + \frac{Ph}{\sigma^2}\right) \tag{1}$$

where b is the channel bandwidth, P is the transmit power, h is the channel gain, and σ is the white noise. The throughput of the link (u_i^t, w_i^t) is expressed as:

$$L_{th}(u_i^t, w_i^t) = b \log_2 \left(1 + \frac{P_{u_i^t}^{w_i^t} A * (3 \times 10^8)^2}{\sigma^2 \left(4\pi f l_{u_i^t}^{w_i^t}\right)^2}\right) \tag{2}$$

where A is the antenna gain, f is the carrier frequency, $P_{u_i^t}^{w_i^t}$ is the transmission power of the link (u_i^t, w_i^t), and $l_{u_i^t}^{w_i^t}$ is the distance between the task u_i^t and the edge server w_i^t.

Task transmission delay is defined as the time required to transmit a task from the user device to the edge server. It is calculated as follows:

$$T_{tr}(u_i^t) = \frac{d_i^t}{L_{th}(u_i^t, w_i^t)} \tag{3}$$

where $L_{th}(u_i^t, w_i^t)$ represents the throughput of the link (u_i^t, w_i^t) between the user device and the edge server.

B. Computation Delay

The computation delay of the task depends on the processing capacity of the edge server. Under ideal failure-free conditions, the computation delay for edge server e_j to successfully complete task u_i^t is expressed as:

$$\zeta_i^t = \frac{c_i^t}{f_i^t} \tag{4}$$

where c_i^t represents the number of CPU cycles required for task u_i^t, and f_i^t represents the computational resources allocated to task u_i^t by edge server e_j.

However, edge server failures may occur during the task offloading process, leading to increasing task completion time. Therefore, investigating the edge server failure is of significant importance for enhancing the safety and reliability of the automobile manufacturing system.

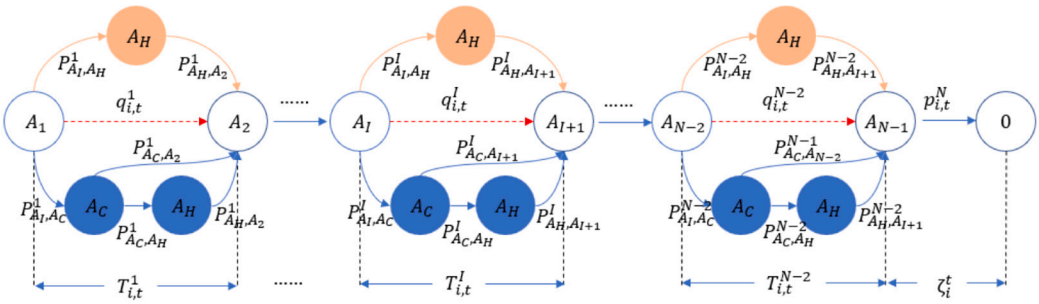


Fig. 2. Markov chain of the edge server failure model.

To analyze the failure of edge servers during the task offloading process, state k ($k = 1, 2, \ldots$) is defined as the k-th execution of the task on the edge server, while state 0 denotes that the task has been completed. During the k-th execution, if the hardware or container failure occurs within the time interval ζ_i^t, i.e., if a failure happens before the task is completed, the task computation will be terminated immediately. The random variables $F_{i,t}^{(k,A_H)}$ and $R_{i,t}^{(k,A_H)}$ denote the hardware failure occurrence time and recovery time of edge server e_j during the k-th execution, respectively. $F_{i,t}^{(k,A_C)}$ and $R_{i,t}^{(k,A_C)}$ denote the failure occurrence time and recovery time of the v-th container e_j^v of edge server e_j during the k-th execution, respectively.

Considering the stability of the edge server, the hardware and container failures of edge servers are modeled as random events following a Poisson process, and the recovery time of these failures is assumed to follow an exponential distribution [24]. The process of edge server e_j executing task u_i^t is described by the Markov chain, as illustrated in Fig. 2. In the model, the state subset for the k-th execution is denoted as $\phi_k = (A_k, A_H, A_C, A_{k+1})$, where A_k represents the k-th execution state and A_{k+1} represents the transition to the $(k+1)$-th execution. A_H and A_C represent the hardware failure state and container failure state, respectively. When a hardware failure occurs, the edge server shuts down, causing all running services to be interrupted. After the edge server is restarted, all deployed containers will be restarted as well. The container failure only affects the tasks running on the corresponding container, while other containers continue to operate normally. Once the failed container is recovered, the affected tasks are re-executed.

Assume that task u_i^t is offloaded to the v-th container e_j^v of edge server e_j for execution. During the k-th execution, if a hardware failure occurs in edge server e_j with a failure rate θ, the probability that task u_i^t fails due to the hardware failure can be expressed as:

$$q_{i,t}^{(k,A_H)} = \Pr(F_{i,t}^{(k,A_H)} < \zeta_i^t + t_{i,t}^k) = 1 - e^{-\theta(\zeta_i^t + t_{i,t}^k)} \tag{5}$$

where $t_{i,t}^k = \max\{t_1, t_2, \ldots, t_c\}$, and t_v represents the execution time of existing tasks on container e_j^v when task u_i^t starts.

Assuming that the container failure follows a Poisson process with a failure rate of λ, the probability that task u_i^t fails due to the container failure is expressed as:

$$q_{i,t}^{(k,A_C)} = \Pr(F_{i,t}^{(k,A_C)} < \zeta_i^t) = 1 - e^{-\lambda\zeta_i^t} \tag{6}$$

Task execution may be interrupted by the hardware or container failure of the edge server. When a failure occurs, the task execution time before interruption is defined as: $F_{i,t}^k = \min(F_{i,t}^{(k,A_C)}, F_{i,t}^{(k,A_H)} - t_{i,t}^k)$ where $F_{i,t}^k$ is a continuous random variable with a range of $(0, \zeta_i^t)$. The failure probability of task u_i^t during its k-th execution is:

$$q_{i,t}^k = \Pr(0 < F_{i,t}^k < \zeta_i^t) = e^{-\theta t_{i,t}^k} - e^{-(\lambda+\theta)\zeta_i^t - \theta t_{i,t}^k} \tag{7}$$

Thus, the probability that task u_i^t completes successfully during its k-th execution is $p_{i,t}^k = 1 - q_{i,t}^k$. When the k-th execution of task u_i^t has failed, $P_{i,t}^{(k,A,B)}(x)$ denotes the one-step transition probability of the edge server transitioning from state A to state B within the time interval x, where $A, B \in \phi_k$. All expressions for the state transition probabilities are provided by Eq. (8)–(12):

$$P_{i,t}^{(k,A_k,A_H)}(x) = \frac{\theta}{\lambda+\theta}\left(e^{-\theta t_{i,t}^k} - e^{-(\lambda+\theta)x - \theta t_{i,t}^k}\right) \tag{8}$$

$$P_{i,t}^{(k,A_k,A_C)}(x) = \frac{\lambda}{\lambda+\theta}\left(e^{-\theta t_{i,t}^k} - e^{-(\lambda+\theta)x - \theta t_{i,t}^k}\right) \tag{9}$$

The recovery time for the hardware failure and container failure are assumed to follow the exponential distribution with the recovery rate η and μ, respectively. Hardware failures may still occur on edge servers after a container failure. When $F_{i,t}^{k,A_C} + R_{i,t}^{k,A_C} \le F_{i,t}^{k,A_H} - t_{i,t}^k$, hardware failures do not occur during container recovery process. In this case, the probability that the system can successfully transition to the $(k+1)$-th task execution state is expressed as:

$$P_{i,t}^{(k,A_C,A_{k+1})}(x) = \frac{\lambda\mu}{(\lambda+\theta)(\theta+\mu)}\left(1 - e^{-(\lambda+\theta)\zeta_i^t}\right)\left(1 - e^{-(\theta+\mu)x}\right)e^{-\theta t_{i,t}^k} \tag{10}$$

When $F_{i,t}^{k,A_C} + R_{i,t}^{k,A_C} > F_{i,t}^{k,A_H} - t_{i,t}^k$, a hardware failure occurs during the container recovery process, and the probability that the system transitions to the hardware failure state A_H is expressed as:

$$P_{i,t}^{(k,A_C,A_H)}(x) = \frac{\mu\theta e^{-\theta t_{i,t}^k}}{(\lambda+\theta)(\lambda-\mu)}\left(1 - e^{-(\lambda+\theta)\zeta_i^t}\right)\left(e^{(\lambda-\mu)x} - 1\right) \tag{11}$$

When the edge server is in the hardware failure state A_H, the probability of recovering from the hardware failure and transitioning to the $(k+1)$-th state is:

$$P_{i,t}^{(k,A_H,A_{k+1})}(x) = \Pr(R_{i,t}^{(k,A_H)} < x \mid X_0 = A_H) = 1 - e^{-\eta x} \tag{12}$$

When the v-th container e_j^v of edge server e_j executes task u_i^t, the time required to resume operation after the k-th failure is denoted as $T_{i,t}^n$. Its cumulative distribution function is expressed as:

$$\begin{aligned} F_{i,t}^k(x) = {} & P_{i,t}^{(k,A_k,A_H)}(x) * P_{i,e_j^v,t}^{(k,A_H,A_{k+1})}(x) \\ & + P_{i,t}^{(k,A_k,A_C)}(x) * P_{i,t}^{(k,A_C,A_{k+1})}(x) \\ & + P_{i,t}^{(k,A_k,A_C)}(x) * P_{i,t}^{(k,A_C,A_H)}(x)\, P_{i,t}^{(k,A_H,A_{k+1})}(x) \end{aligned} \tag{13}$$

where the symbol * represents the Stieltjes convolution of two functions.

Each task execution is considered as an independent event. If a task execution fails, it restarts from the same state. Assume that completing a task requires $N \in \mathbb{N}$ executions, where the N-th execution is successful and the preceding $N-1$ executions failed. The random variable N follows a non-homogeneous geometric distribution, with the probability mass function defined as:

$$p_{i,t}(N) = \Pr(k = N) = p_{i,t}^N \prod_{k=1}^{N-1} q_{i,t}^k \tag{14}$$

The total computation delay L_i^t of task u_i^t is:

$$L_i^t = \sum_{k=1}^{N-1} T_{i,t}^k + \zeta_i^t \tag{15}$$

C. Queuing Delay

Each edge server is equipped with c containers, allowing multiple tasks to be processed concurrently. When a new task arrives, if there

is an idle container, it will be immediately assigned to the container without queuing delay. Otherwise, the new task will enter the First-Come-First-Served (FCFS) queue of the edge server. The queuing delay of task u_i^t depends on the queuing delay of the previous task in the queue and operational state of each container when it starts executing. The queuing delay be expressed as follows:

$$T_q(u_i^t) = T_q^{pre} + \min\{T_j^1, T_j^2, \ldots, T_j^c\} \tag{16}$$

where T_q^{pre} represents the queuing delay of the tail task u_{pre} before the new task enters the queue, and T_j^v represents the remaining execution time of tasks in the v-th container when task u_{pre} starts execution.

3.3. Task offloading optimization objective

The section will further study the optimization objectives and corresponding evaluation indicators of task offloading. To characterize the potential risk of extreme delays in task offloading (i.e., the right tail of the delay distribution), CVaR is introduced as a metric in the paper. CVaR is able to effectively capture the risk of extreme delays [25], and its minimization formula can be easily incorporated into optimization models to provide stable and accurate risk assessments. The lower the CVaR, the lower the probability of extreme delays and the lower the risk to the system. At a given significance level α, the CVaR of a random variable X is defined as:

$$\mathrm{CVaR}_\alpha(X) = \frac{1}{1-\alpha}\int_\alpha^1 \mathrm{VaR}_\epsilon(X)\, d\epsilon \tag{17}$$

where $\mathrm{VaR}_\epsilon(X)$ is the Value at Risk (VaR), representing the quantile of the random variable X at confidence level ϵ.

When the completion time $T_f(u_i^t)$ of task u_i^t does not exceed its deadline D_i^t, the task is considered completed on time; otherwise, it is considered to be overtime. The total number of tasks completed on time is U:

$$U = \sum_{t=1}^{t_{\max}} \sum_{i=1}^{n} F(T_f(u_i^t) - D_i^t) \tag{18}$$

where the indicator function $F(x)$ is defined as:

$$F(x) = \begin{cases} 0, & x > 0 \\ 1, & x \le 0 \end{cases} \tag{19}$$

The task completion rate is an important metric to measure the reliability of IIoT systems and is defined as the ratio of tasks completed on time to the total number of tasks N. This paper simultaneously optimizes the CVaR and the task completion rate during task offloading to improve the reliability of the system. Based on the above analysis, the optimization problem is defined as:

$$\min \quad a_1 \cdot \mathrm{CVaR}_\alpha + a_2 \cdot \left(1 - \frac{U}{N}\right) \tag{20}$$

subject to:

$$a_1 + a_2 = 1, \quad a_1 > 0, \quad a_2 > 0 \tag{21}$$

$$F(T_f(u_i^t) - D_i^t) = \begin{cases} 0, & T_f(u_i^t) - D_i^t > 0 \\ 1, & T_f(u_i^t) - D_i^t \le 0 \end{cases} \tag{22}$$

$$t \in \{1, 2, \ldots, t_{\max}\}, \quad i \in \{1, 2, \ldots, n\} \tag{23}$$

where a_1 and a_2 are weight parameters for balancing CVaR and the task timeout rate.

4. ESFMTO strategy based on SAC-BNN

The probability distribution of task completion time plays a critical role in optimizing task offloading decisions. To deal with the high complexity and the dynamic uncertainty of environments in IIoT, BNN is employed to model the probability distribution of task completion time and compute the CVaR based on the obtained distribution in real-time. CVaR is incorporated into the reward function of the SAC-BNN algorithm, enhancing the reliability and robustness of the offloading strategy. Additionally, a perturbation generation network is designed to apply appropriate perturbations to input states, improving the adaptability of the algorithm to uncertain environments. Fig. 3 illustrates the scheme of ESFMTO.

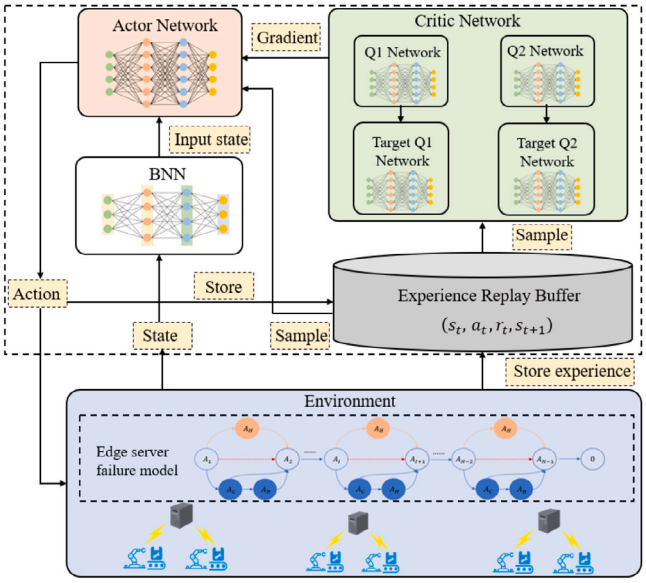


Fig. 3. The scheme of ESFMTO.

4.1. BNN-based task completion time learning

BNN [26] integrates Bayesian probability theory into the neural network framework, which effectively captures the uncertainty by modeling the weights and bias parameters of the network as probability distributions. Compared with traditional deterministic neural networks, BNN can not only provides robust confidence intervals and risk assessments, but also exhibits greater robustness and adaptability. In traditional neural networks, model parameter ω is considered as fixed value and trained by optimization methods such as gradient descent. In BNN, the model parameters are modeled as random variables whose uncertainty is described by the prior distribution $p(\omega)$ and the posterior distribution $p(\omega \mid D)$. $p(\omega)$ reflects the initial assumption about the parameters, while $p(\omega \mid D)$ is updated by combining the observed data D and the prior distribution.

Due to the extremely high computational complexity of directly solving the posterior distribution $p(\omega \mid D)$ in the high-dimensional parameter space, Variational Inference (VI) [27] is commonly used for approximation. VI introduces an easy-to-compute approximate distribution $q_\theta(\omega)$ to approximate the true posterior distribution $p(\omega \mid D)$. The objective is to reduce the difference between these two distributions by minimizing the Kullback–Leibler (KL) divergence, an asymmetric measure of the difference between two probability distributions, which is formulated as follows:

$$D_{\mathrm{KL}}(q_\theta(\omega) \,\|\, p(\omega \mid D)) = \mathbb{E}_{q_\theta(\omega)}\left[\log \frac{q_\theta(\omega)}{p(\omega \mid D)}\right] \tag{24}$$

According to Bayes' theorem, the posterior distribution $p(\omega \mid D)$ is expressed as:

$$p(\omega \mid D) = \frac{p(\omega)\, p(D \mid \omega)}{p(D)} \tag{25}$$

where $p(D \mid \omega)$ denotes the likelihood of the observed data under the given network parameter ω, $p(\omega)$ is the prior distribution of the

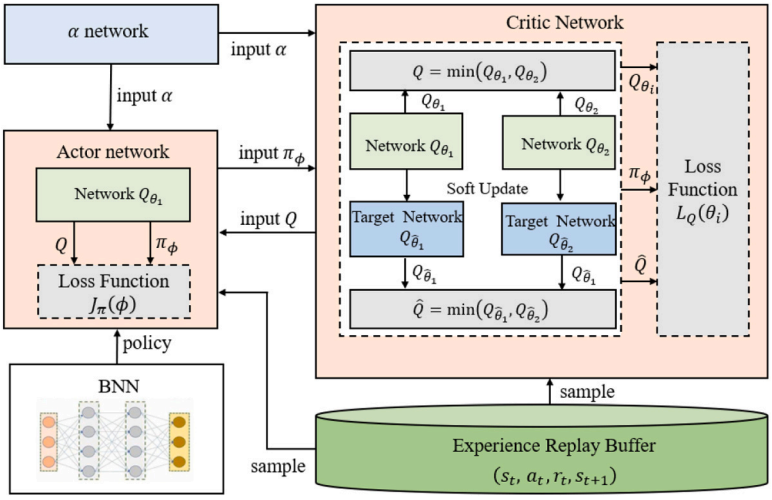


Fig. 4. The structure of the SAC-BNN algorithm.

parameters, and $p(D)$ is the marginal likelihood. Substituting (25) into (24), we can obtain:

$$D_{\mathrm{KL}}(q_\theta(\omega) \parallel p(\omega \mid D)) = \mathbb{E}_{q_\theta(\omega)}[p(D)] - \mathbb{E}_{q_\theta(\omega)}\left[\log \frac{p(\omega)p(D \mid \omega)}{q_\theta(\omega)}\right] \tag{26}$$

where the second item is called Evidence Lower Bound (ELBO), and it is expressed as:

$$L = \mathbb{E}_{q_\theta(\omega)}\left[\log \frac{p(\omega)p(D \mid \omega)}{q_\theta(\omega)}\right] \tag{27}$$

By maximizing the ELBO to find the optimal parameter θ^*, the approximate distribution $q_\theta(\omega)$ closely approximates the true posterior distribution $p(\omega \mid D)$:

$$\theta^* = \arg\max_\theta \mathbb{E}_{q_\theta(\omega)}\left[\log \frac{p(\omega)p(D \mid \omega)}{q_\theta(\omega)}\right] \tag{28}$$

The task completion time T is considered a random variable whose distribution is learned by the BNN. To accurately describe the uncertainty of model parameters, both the prior distribution and approximate posterior distribution of the weights and bias parameters of BNN use t-distribution:

$$p(\omega) \sim \mathrm{StudentT}(\nu_p, \mu_p, \sigma_p^2) \tag{29}$$

$$q(\omega) \sim \mathrm{StudentT}(\nu_q, \mu_q, \sigma_q^2) \tag{30}$$

where ν_p, ν_q denote the freedom degrees of the prior and approximate posterior distributions, respectively. μ_p, μ_q are the means, and σ_p^2, σ_q^2 are the variances.

Compared to the commonly used Gaussian distribution, the t-distribution has heavier tail properties, which means it is more flexible and precise in modeling tail delays. By modeling parameter uncertainty as the t-distribution, BNN can more accurately capture the tail features of the task completion time distribution, effectively addressing the potential delay risk during task offloading.

4.2. SAC-BNN algorithm

To deal with the complex optimization problem in task offloading, the paper leverages the powerful capability of DRL [28] in high-dimensional decision space and proposes a task offloading algorithm based on SAC and BNN, namely SAC-BNN. Specifically, the task offloading problem is modeled as a Markov Decision Process (MDP) and the SAC-BNN algorithm is used to solve it. By incorporating CVaR into the reward function of SAC, the effective management of the extreme delay risk is achieved, improving the robustness and performance of the task offloading strategy. Fig. 4 illustrates the structure of SAC-BNN.

The task offloading problem is modeled as an MDP, and the specific definitions of the state space, the action space, and the reward function are as follows.

(a) State space

The state space $S_t = (D, W, E, Q)$ describes the state of the system in a particular time slot, where $D = (l_{(u_i,e_1)}, l_{(u_i,e_2)}, \ldots, l_{(u_i,e_m)})$ represents the physical distance between the user and each edge server. $W = \left\{\mathrm{Deadline}_i \ \{d_i^t\}_{j=1}^{k_{(u_i)}}, \ \{\zeta_i^t\}_{j=1}^{k_{(u_i)}}\right\}_i^{k_t}$ denotes the set of specific information of the tasks, where $\mathrm{Deadline}_i$ is the deadline of task u_i^t, d_i^t is the size of task u_i^t, ζ_i^t is the number of CPU cycles required for task u_i^t. $E = (\mathrm{Server}_1, \mathrm{Server}_2, \ldots, \mathrm{Server}_m)$ denotes the set of edge server status information, including the CPU power, failure status, container failure condition, server failure rate and container failure rate of each edge server. $Q = (q_{(e_1)}, q_{(e_2)}, \ldots, q_{(e_m)})$ denotes the queue length of all edge servers.

(b) Action space

The action space defines all possible offloading actions of the agent at time point t. For task u_i^t, its offloading action $a_i^t \in \{e_1, e_2, \ldots, e_m\}$ indicates which edge server the agent chooses to offload the task to.

(c) Reward function

The reward function evaluates the immediate feedback of the agent after executing an action, guiding the strategy to optimize towards minimizing risk and maximizing task on-time completion rate. The paper comprehensively considers the extreme delay risk and task completion time in the reward function. Specifically, the reward function consists of the following components:

- Delay risk: CVaR is used to punish potential delay losses in extreme scenarios, encouraging the agent to make low-risk offloading decisions.
- Task completion time: A positive reward is granted if the task is completed on time, and a negative penalty is given according to the degree of delay if the task exceeds its deadline. Therefore, the reward function is defined as:

$$R = -\alpha_1 \cdot \mathrm{CVaR}_\alpha + \alpha_2 \cdot I(u_i^t) - \alpha_3 \cdot (1 - I(u_i^t)) \cdot (T_f(u_i^t) - D_i^t) \tag{31}$$

where α_1, α_2, and α_3 are weight parameters used to balance delay risk, reward, and penalty. $I(u_i^t) = F(T_f(u_i^t) - D_i^t)$ is an indicator function to determine whether the task is completed within the deadline. Algorithm 1 describes the task offloading algorithm based on SAC-BNN.

Considering all nested loops and operations, the total time complexity of the algorithm is expressed as $O(N \cdot t_{\max} \cdot U \cdot (n_a + T))$, where N denotes the total number of training episodes, $t_{\max}$ denotes

Algorithm 1 Task Offloading Algorithm Based on SAC-BNN

Input: user set U, edge server set E, task offloading environment, actor network, BNN network
Output: task offloading decision $\{w_1^t, w_2^t, \ldots, w_n^t\}_{t=1}^{t_{\max}}$

Initialize the task offloading environment, experience replay pool B, actor network parameters ϕ, and task completion time set T;
for $Episode = 1$ to N **do**
  for $t = 1$ to $t_{\max}$ **do**
    for each $u \in U$ **do**
      Obtain the state s_t of task u_i^t before processing;
      Input s_t into the actor network to obtain the action distribution $\pi_\phi(a \mid s)$ and sample the action $a_t \sim \pi_\phi(\cdot \mid s_t)$;
      Calculate the reward function r_t based on the calculated CVaR and Eq. (31);
      Get the updated state s_t';
      Store transition (s_t, a_t, r_t, s_t') into the replay buffer B;
      Store the completion time $T_f(u_i^t)$ of task u_i^t into T;
      Calculate the distribution of the set T according to Eq. (28) and compute CVaR in real time;
    end for
  end for
end for

the number of time steps per episode, and U denotes the number of user tasks. n_a denotes the number of parameters in the actor network, T denotes the size of the task completion time set. Integrating the BNN with the SAC does indeed increase computational complexity, thereby extending the training time. However, since SAC-BNN is deployed after offline training, the additional training cost has a negligible impact on the performance of real-time task offloading. On the contrary, this combination enhances the reliability and robustness of the system, ensuring that accurate and timely offloading decisions can be made in IIoT.

4.3. The network structure and parameters updating of SAC

In the SAC framework, the algorithm is determined by the actor network whose output is the action probability distribution $\pi_\phi(a \mid s)$ for a given state s_t, where ϕ is a parameter of the actor network. The critic network is used to evaluate the Q-value of the state–action pairs, and there are two Q networks (with parameters θ_1 and θ_2, respectively) and their corresponding two target Q networks (with parameters $\hat{\theta}_1$ and $\hat{\theta}_2$ respectively).

The actor network outputs the action distribution $\pi_\phi(\cdot \mid s)$ based on the current state s_t at each time step t, and obtains the action $a_t \sim \pi_\phi(\cdot \mid s_t)$ through sampling. To encourage exploration, SAC incorporates the entropy regularization term into the loss function of the actor network:

$$J_\pi(\phi) = \mathbb{E}_{a_t \sim \pi_\phi}\left[\alpha \log \pi_\phi(a_t \mid s_t) - Q\right] \tag{32}$$

where $Q = \min(Q_{\theta_1}(s_t, a_t), Q_{\theta_2}(s_t, a_t))$, α is the temperature parameter used to adjust the influence of the entropy regularization term. The gradient of the actor network parameter ϕ is updated as:

$$\phi \leftarrow \phi - \beta \nabla_\phi J_\pi(\phi) \tag{33}$$

where β is the learning rate of the actor network. The critic network comprises two Q-networks $Q_{\theta_1}(s_t, a_t), Q_{\theta_2}(s_t, a_t)$ and two corresponding target Q-networks $Q_{\hat{\theta}_1}(s_t, a_t), Q_{\hat{\theta}_2}(s_t, a_t)$, which are used to estimate the value of state–action pairs. The target value is expressed as:

$$y = r_t + \gamma \mathbb{E}_{a_{t+1} \sim \pi_\phi}\left[\min_{i=1,2} Q_{\hat{\theta}_i}(s_{t+1}, a_{t+1}) - \alpha \log \pi_\phi(a_{t+1} \mid s_{t+1})\right] \tag{34}$$

The parameters θ_1 and θ_2 of the Q-network are updated by minimizing the following Mean Squared Error (MSE) loss function:

$$L_Q(\theta_i) = \left(Q_{\theta_i}(s_t, a_t) - y\right)^2, \quad i = 1, 2 \tag{35}$$

The gradients of θ_1 and θ_2 are updated as:

$$\theta_i \leftarrow \theta_i - \rho_i \nabla_{\theta_i} L_Q(\theta_i), \quad i = 1, 2 \tag{36}$$

The target Q-network parameters $\hat{\theta}_1$ and $\hat{\theta}_2$ are updated regularly through the soft update method to maintain the stability of the target values. The specific parameter update method for $\hat{\theta}_1$ and $\hat{\theta}_2$ is as follows:

$$\hat{\theta}_i \leftarrow \tau_i \theta_i + (1 - \tau_i)\hat{\theta}_i, \quad i = 1, 2 \tag{37}$$

where τ_1 and τ_2 are soft update parameters.

Assuming the batch size is B, the time complexity of Algorithm 2 is expressed as $O(N \cdot B \cdot (n_p + n_a + n_q))$, where n_p represents the number of parameters in the perturbation generation network and n_q represents the number of parameters in each Q network.

4.4. Perturbation generation network

To enhance the robustness of SAC-BNN in uncertain IIoT environments, a perturbation generation network is proposed, based on the theoretical principle of adversarial training. This network applies small perturbations to the input state, thereby enabling the SAC-BNN algorithm to learn policies that can effectively cope with environmental variations and potential failures. Specifically, the perturbation generation network generates perturbed states $\hat{s}_t$ by adding an appropriate perturbation δ to the original state s_t:

$$\hat{s}_t = s_t + \delta \tag{38}$$

where the perturbation δ satisfies the norm constraint: $\|\delta\| \le \varepsilon$. Here, ε is a predefined small positive constant.

This constraint ensures that the perturbation amplitude remains within a reasonable bound, preventing unrealistic state deviations. The perturbation δ aims to maximize the policy loss to promote the robustness of learned policies under worst-case scenarios. The perturbation generation network is trained by minimizing the following loss function:

$$L_{\text{pgn}} = -J_\pi(s_t + \delta) \tag{39}$$

The perturbation generation network addresses the following constrained optimization problem:

$$\delta^* = \arg\max_{\|\delta\| \le \varepsilon} J_\pi(s_t + \delta) \tag{40}$$

The optimization problem is solved using a projected gradient ascent method defined by the iterative update rule:

$$\delta \leftarrow \text{Proj}_\varepsilon\left(\delta + \vartheta \nabla_\delta J_\pi(s_t + \delta)\right) \tag{41}$$

where ϑ denotes the step size, and the projection operator $\text{Proj}_\varepsilon(\cdot)$ ensures the perturbation remains within the allowed norm constraint:

$$\text{Proj}_\varepsilon(\delta) = \varepsilon \cdot \frac{\delta}{\max(\varepsilon, \|\delta\|)} \tag{42}$$

In the perturbation generation network, small perturbations are introduced to the inputs to assess and enhance the stability and reliability of the learning model. The network generates perturbations that simulate worst-case scenarios for the policy network, forcing SAC-BNN to adapt to highly challenging inputs and thereby significantly boosting its robustness. As a result, the learned policy becomes more resilient to the input uncertainties and perturbations that frequently occur in IIoT scenarios, ensuring stability and reliability in practical deployments. Algorithm 2 describes the network parameter updating process for SAC.

Algorithm 2 Updating the Network Parameters of SAC

Input: user set U, edge server set E, task offloading environment, replay buffer B
Output: critic and actor network parameters $\theta_1, \theta_2, \hat{\theta}_1, \hat{\theta}_2$, and ϕ

```
Initialize task offloading environment, critic and actor network parameters θ1, θ2, θ̂1, θ̂2, and φ;
for Episode = 1 to N do
    for each batch b ∈ B do
        Input the state s_t into the perturbation generation network to obtain the perturbed state ŝ_t;
        Calculate the target values y1 and y2 for the two Q-networks according to Eq. (34);
        Calculate the loss function for the actor network and the two Q-networks according to Eqs. (32) and (35);
        Update the actor and critic networks according to Eqs. (33) and (36);
        Regularly update the two target Q-networks according to Eq. (37);
    end for
    Clear replay buffer B;
end for
```

Table 2
Parameter information table.

Parameter type	Value	Parameter type	Value
Number of edge servers	20	Container failure rate	0.03–0.09
Number of users	[50–70]	Confidence level	75%–95%
Hardware recovery rate	2	Noise power	10^{-10} W
Container recovery rate	3	CPU frequency parameter	2 GHz
Hardware failure rate	0.02–0.05	Channel gain	40 dB

5. Experiments and evaluation

In this section, the proposed SAC-BNN algorithm is evaluated. First, the detailed experimental setup is presented. Then the performance of SAC-BNN is evaluated from multiple dimensions and compared with the baseline algorithms in terms of on-time task completion rate and CVaR. Finally, the experimental results are analyzed.

5.1. Experimental setup

The algorithm uses a neural network with a fully connected layer structure, where both actor and critic networks contain two hidden layers, with 512 and 128 neurons in the hidden layer, respectively. Relu is used as the activation function for the output. The training process uses a mini-batch size of 64. The learning rates of the actor network and the critic network are both set to 10^{-4}, and the learning rates of the temperature parameter network and the perturbation generation network are set to 10^{-4} and 10^{-5}, respectively. The soft update τ of the target network is set to 0.005, the penalty factor γ is set to 0.9, and the Adam optimizer is used to optimize the neural network. Other parameters are detailed in Table 2.

To verify the effectiveness of the proposed algorithms, the following five task offloading methods are selected as the baseline algorithms:

- DDQN [29]: DDQN is a widely adopted and well-established value-based DRL method that mitigates overestimation bias through its double network architecture.
- A2C [30]: A2C, as a representative of Actor-Critic methods, is suitable for distributed decision-making scenarios and offers high computational efficiency.
- PG [31]: PG directly optimizes policy parameters, leveraging the strengths of policy-based approaches in decision-making.
- DDPG [32]: DDPG combines the Actor-Critic framework with continuous action spaces, utilizing experience replay and target networks to stabilize training.
- SAC [17]: The SAC-BNN algorithm proposed in the paper is based on the SAC framework.
- Risk-Sensitive RL (RS-RL) [22]: Risk-Sensitive RL integrates risk constraints into its learning process to meet strict deadline requirements.

In the experiments, a clear and strict deadline is set for each task, and the system is required to complete task within the prescribed timeframe to simulate the stringent real-time response requirements of IIoT. The real-time responsiveness of the system is assessed by calculating the on-time completion rate. A high on-time completion rate indicates that the system performs well in meeting real-time demands. Moreover, the CVaR is employed to analyze the extreme delay risk of the system, which ensures the stability and reliability in real-time task processing.

5.2. Evaluation of performance of SAC-BNN

(1) Convergence Performance

Fig. 5 illustrates the variations in the loss values of the two Q-value networks in SAC-BNN. The losses of the two Q-value networks fluctuated at the beginning of training, but stabilizes at a low level after about 2000 steps, indicating that the losses gradually approach the optimum. In general, the stability of the Q network loss demonstrates that SAC-BNN achieves robust convergence performance.

(2) Reward Value

Fig. 6 shows the trends in average episode rewards across different algorithms during training. Presenting reward values allows for an intuitive comparison of each algorithm's learning performance and stability throughout the training process. Notably, SAC-BNN exhibits exceptional performance during training, achieving rapid convergence and maintaining the high and stable average reward value. In contrast, SAC and A2C achieve slightly lower final reward values compared to SAC-BNN, but they still converge relatively quickly to the high reward level. RS-RL does not converge in the first 200 rounds, but it successfully converges to a high reward value. The DDPG algorithm begins to converge around the 250th episode and continues to fluctuate thereafter, indicating instability in its training process. DDQN and PG show comparatively weaker performance and their average reward values at the end of training are significantly lower than those of the other algorithms. Different from SAC-BNN, these algorithms do not consider the extreme delay risk in their training processes, resulting in low reward values.

(3) Probability Density Function Distribution of Delay

As shown in Fig. 7, the Probability Density Function (PDF) distributions of delay for six algorithms are compared. It can be observed that SAC-BNN achieves the highest probability density in the low-delay range and peaks around a delay of 2.5, indicating that more tasks are completed at the same latency compared to the other algorithms. As the delay increases, the distribution curve of SAC-BNN declines rapidly with a shorter tail, which demonstrates that its delay distribution is concentrated and that the extreme delay risk is lower. Although SAC, A2C, and RS-RL perform well in the low-delay region, their peaks are both lower than those of SAC-BNN and occur further to the right, implying relatively long completion delays for most tasks. The distribution of DDQN and PG are more dispersed with longer tails. While DDPG also shows a relatively high peak near 2.5, its overall distribution remains broad. The superior performance of SAC-BNN is attributed to the real-time calculation of CVaR, which measures the potential extreme delay risk during the task offloading process.

(4) Evaluation with Different Numbers of Users

In Fig. 8(a), the on-time task completion rate of different algorithms under varying numbers of users is compared. As the number of users increases, the on-time task completion rate of all algorithms decreases, which indicates that high load prolongs the task completion time. Among them, the performance of DQN and PG algorithms declines

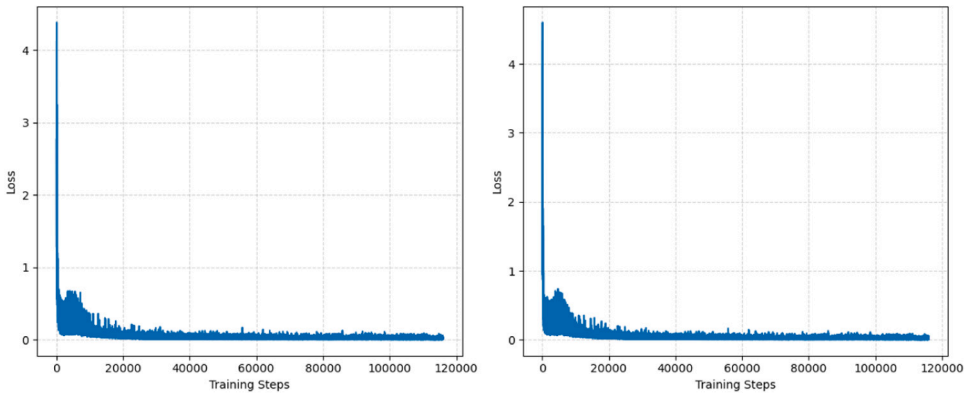


Fig. 5. Convergence performance of the two Q networks.

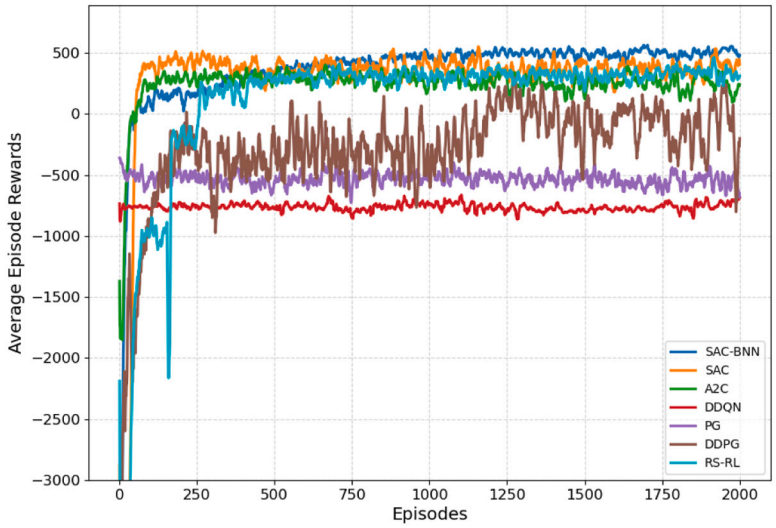


Fig. 6. Reward values of different algorithms.

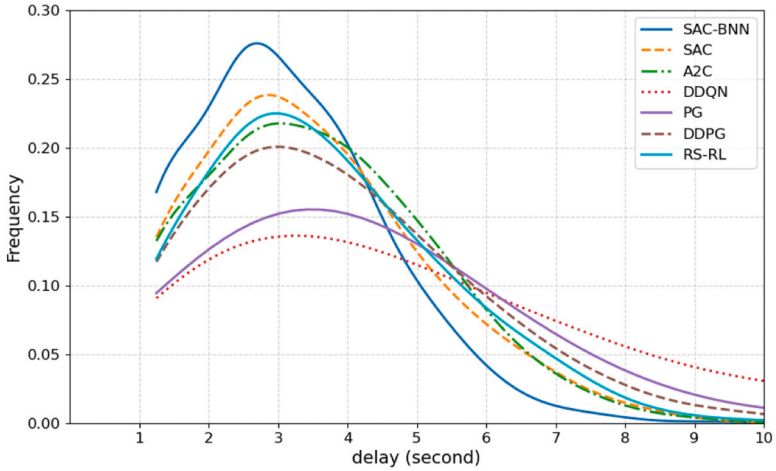
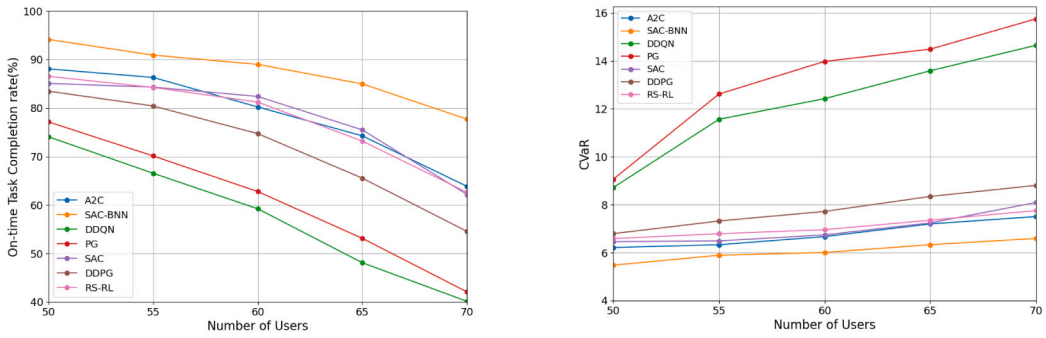


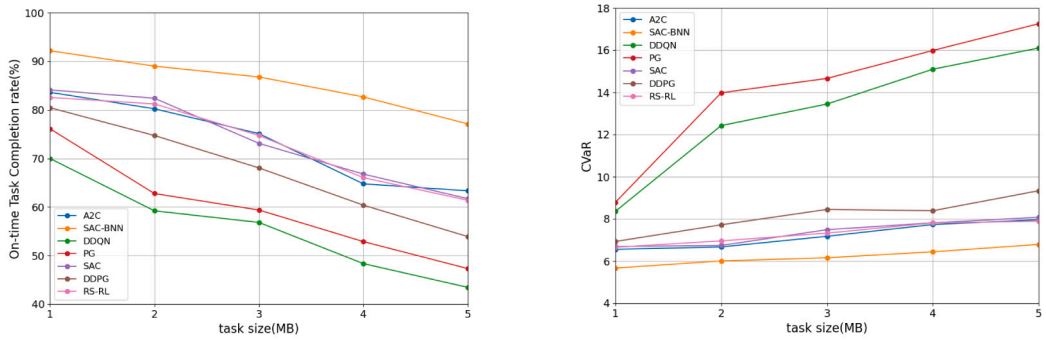
Fig. 7. Distribution of the task completion time.

most significantly. SAC-BNN shows the smallest drop and maintains a high level of performance even when the number of users reaches 70. Specifically, at 70 users, the on-time task completion rate of SAC-BNN exceeds those of A2C, RS-RL, SAC, DDPG, PG, and DDQN by 13.9%, 15.1%, 15.5%, 23.1%, 35.6%, and 37.6%, respectively, highlighting its strong robustness. Fig. 8(b) presents the CVaR of different algorithms when the number of users increases. CVaR is used to measure the extreme delay risk of the system in task offloading. The smaller the CVaR, the lower the extreme delay risk of the system, and the higher its reliability. As the number of users increases, the CVaR of all algorithm rises, indicating that the extreme delay risk of the system increases. SAC-BNN always performs best in CVaR under different numbers of users. The superior performance of SAC-BNN is attributed to the fact that it integrates CVaR into the reward function of DRL. By calculating



(a) (b)

Fig. 8. Comparison of different numbers of users.



(a) (b)

Fig. 9. Comparison of different task sizes.

CVaR in real time during the task offloading process, SAC-BNN effectively reduces the extreme delay risk of the system, thereby enhancing the system's ability to maintain high real-time responsiveness even under heavy workload scenarios.

(5) Evaluation of Different Task Sizes

Fig. 9(a) compares the on-time completion rate of different algorithms under varying task sizes (1 MB to 5 MB). The result shows that as the task size increases, the on-time completion rate of all algorithm decreases to varying degrees, indicating that larger-scale tasks require more computational resources and the difficulty of completing tasks on time increases. The performance of the DDQN and PG algorithms drops significantly and when the task size is 5 MB, their on-time completion rate drops to 43.46% and 47.33%, respectively. In contrast, SAC-BNN performs more robustly and maintains the on-time completion rate exceeding 70% even when the task size is 5 MB. This is due to that SAC-BNN introduces the perturbation neural network, which adds perturbations to the trained model to better deal with uncertainties in the environment. Fig. 9(b) presents the CvaR of different algorithms under the variation of task size. As the task size increases, the CVaR of all algorithms shows an upward trend. However, SAC-BNN always has the lowest CVaR under each task size condition, and still outperforms other algorithms even in large task of 5 MB. The reason why SAC-BNN can effectively reduce the extreme delay risk in large-scale task environments is that it accurately models the distribution of task completion time through BNN and incorporates CVaR into the decision-making process. By consistently achieving high on-time completion rate and maintaining low CVaR even under large-scale tasks, SAC-BNN exhibits exceptional robustness and adaptability to dynamic real-time constraints.

(6) Impact of Container Failure Rate and Hardware Failure Rate

Fig. 10(a) shows the on-time task completion rates of different algorithms under varying container failure rates. The results show that as the container failure rate increases, the on-time completion rates of all algorithms decrease to different degrees, indicating that the container failure has a significant negative impact on task processing. The DDQN and PG algorithms are particularly affected by the change of the container failure rate. In contrast, SAC-BNN outperforms the other algorithms under varying container failure rates. Fig. 10(b) compares the CVaR of all algorithms as the container failure rate varies. As the container rate increases, the CVaR of all algorithms increases. However, SAC-BNN consistently outperforms at all container failure rate levels, maintaining low CVaR of 7.76 even when the container rate reaches 0.09. The main reason for the low CVaR and high on-time completion rate of SAC-BNN is that it uses a perturbation neural network. The network exposes the algorithm to various possible state during training, enabling it to learn more effective strategies for handling the edge server failure and other uncertain environments. Fig. 10(c) shows the performance of different algorithms in terms of task on-time completion rate under various hard failure rates. The on-time completion rate decreases as the hard failure rate increases. SAC-BNN outperforms the other algorithms under varying hard failure rates. Fig. 10(d) illustrates the CVaR of different algorithms as hard failure rates vary. The SAC-BNN algorithm consistently maintains low CVaR values across all failure rate levels and demonstrates significant advantages even when the hard failure rate reaches 0.05, with 0.72, 0.74, 0.79, 2.19, 7.89, and 9.60 lower than that of RS-RL, A2C, SAC, DDPG, DDQN, and PG, respectively. The reason for the superior performance of SAC-BNN is that it accurately models the distribution of task completion time through BNN and calculates CVaR in real time within the DRL framework. This enables SAC-BNN to effectively sense the server failure and the extreme delay risk, thus minimizing overall system risk. SAC-BNN's real-time decision-making significantly mitigates the impact of uncertainties and failures, thereby ensuring the stability and responsiveness of the system in real-time IIoT applications.

(7) Impact of Task Deadline

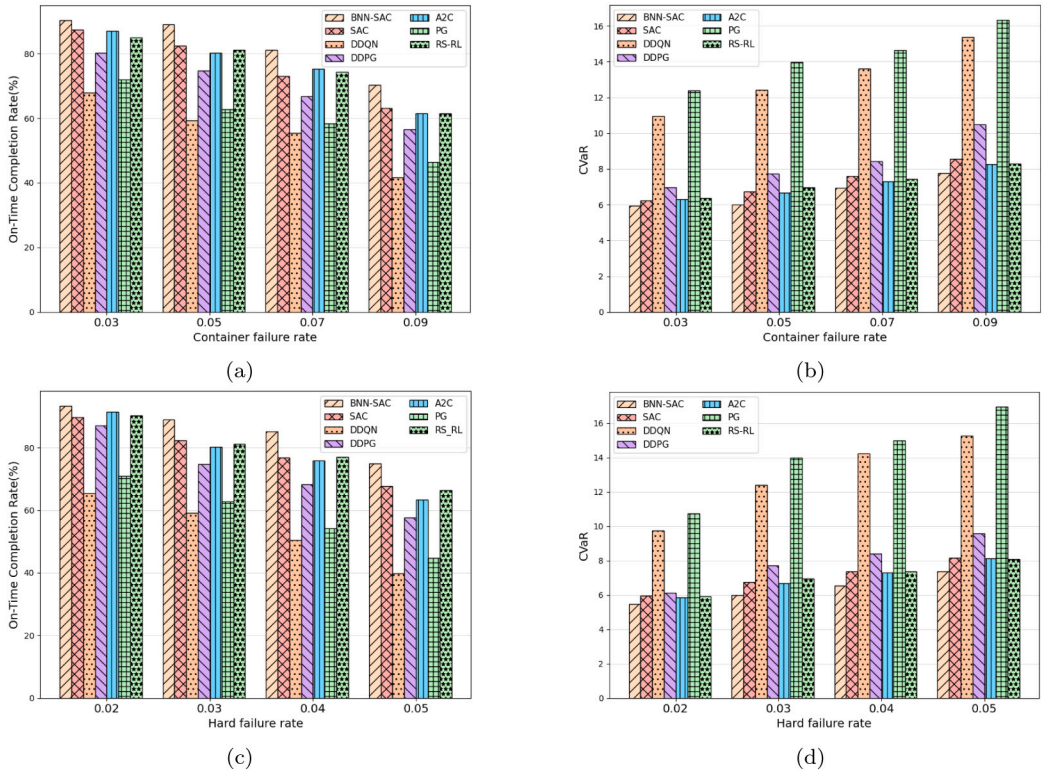


Fig. 10. Comparison of different container failure rates and hardware failure rates.

Fig. 11(a) illustrates the on-time task completion rate of different algorithms as the task deadline increases from 4.0 to 6.0. It can be observed that the performance of all algorithms improves with the relaxation of the task deadline. SAC-BNN consistently achieves the highest on-time task completion rate across all task deadlines. The on-time completion rate of SAC-BNN closes to 100% when the task deadline is 6.0, significantly outperforming the other algorithms. In contrast, PG and DDQN exhibit the lowest performance, with on-time task completion rates staying below 75% even under the longest deadline. Fig. 11(b) presents the CVaR of various algorithms under different task deadlines. SAC-BNN algorithm consistently achieves the lowest CVaR across all task deadlines and maintains a stable CVaR close to 6, indicating minimal sensitivity to variations in task deadlines. In contrast, other algorithms do not consider the extreme delay in their training processes. As a result, when the task deadline is extended, delays in certain tasks may increase yet still within the deadlines. Without CVaR-based feedback in the reward function, these increased delays are not adequately penalized, leading to higher CVaR of other algorithms. In summary, incorporating real-time CVaR risk assessment into the offloading decision process can effectively meet the real-time offloading requirements in IIoT environments.

(8) Impact of Significance Level

Fig. 12 shows the impact of different significance levels (α) on the CVaR for various algorithms. As the significance level increases from 0.75 to 0.95, CVaR exhibits a clear upward trend across all algorithms. SAC-BNN consistently remains the lowest CVaR of all significance levels, with a CVaR of 6.79 even at the highest significance level ($\alpha = 0.95$). A2C, RS-RL, and SAC also perform well, although their CVaR remains higher than that of SAC-BNN. PG and DDQN exhibit the highest CVaR at $\alpha = 0.95$, exceeding 15.5 and 13.5, respectively. Overall, the results highlight the superior performance of SAC-BNN, which is owing to its consideration of CVaR during the task offloading process. SAC-BNN integrates CVaR into the reward function, enabling it to adjust the offloading decision in real time based on the system risk. This enables SAC-BNN to effectively quantify and minimize the extreme delay risk, thereby ensuring the high reliability and responsiveness required for real-time IIoT applications.

5.3. Discussion

Overall, the experimental results fully validate the capabilities of SAC-BNN for real-time task offloading. The experiments indicate that under various conditions, SAC-BNN consistently outperforms the baseline algorithms. Although all experiments are conducted in the simulated environment and have yet to be validated in real industrial environments, the simulation scenarios closely approximate real-world industrial conditions. Specifically, the proposed edge server failure model accounts for both hardware and container failures that occur in practical scenarios. Furthermore, unlike many studies that rely on simplifying assumptions, our consideration of multiple containers per edge server more accurately reflects the complexities encountered in large-scale IIoT environments. Moreover, although experimental validations have primarily been conducted in small-scale scenarios due to resource constraints, the algorithm design has fully accounted for the complexities and high-load challenges inherent in large-scale IIoT deployments. Extensive simulation experiments under varying conditions of user scale, task scale, and failure rates indicate that even under high-load conditions, the SAC-BNN algorithm maintains robust performance and significantly mitigates the extreme delay risk, thereby demonstrating its adaptability in large-scale IIoT environments. However, actual deployments may introduce unforeseen factors that could affect the algorithm's performance. Future research will focus on experimental validations in large-scale IIoT environments to comprehensively assess the performance of the proposed algorithm. Table 3 presents a comparative analysis between the baseline algorithm and the SAC-BNN algorithm.

6. Conclusion

In this paper, a task offloading strategy ESFMTO based on edge server failure model is proposed for the real-time task offloading in

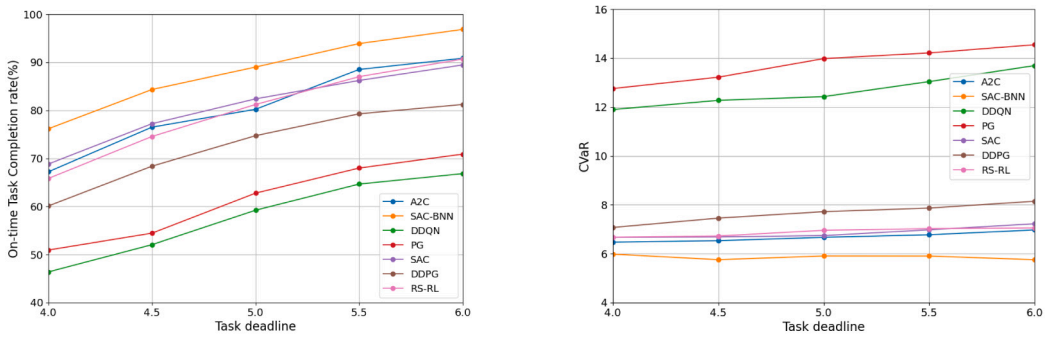


Fig. 11. Comparison of different task deadlines.

Table 3
Comparison of task offloading algorithms.

Algorithm	Advantages	Disadvantages
DDQN [29]	Overestimation bias mitigation; stable learning process	Lack of the delay risk management; limited scalability
A2C [30]	High computational efficiency; suitable for distributed scenarios	Lack of the delay risk management; poor performance under extreme conditions
PG [31]	Direct policy optimization; simple structure	High variance and unstable training performance
DDPG [32]	Relatively stable training performance	Hyperparameter sensitivity; limited robustness under complex and dynamic conditions
SAC [17]	Effective exploration via entropy regularization; stable and efficient learning performance	Lack of the delay risk quantification
RS-RL [22]	Explicit risk consideration; well-suited for applications with strict deadline requirements	Slow and unstable initial convergence
SAC-BNN	Accurate delay uncertainty modeling; explicit extreme delay risk mitigation; stability	Increased training complexity; offline training requirement

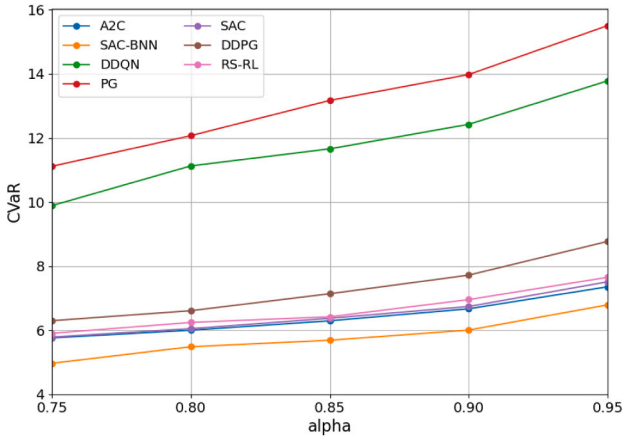


Fig. 12. Comparison of different significance levels.

IIoT. Firstly, the edge server failure model for IIoT is designed, and the interaction mechanism during both the occurrence and recovery of failures is analyzed in depth. On the basis, the paper establishes a task offloading optimization model with system delay and CVaR as dual optimization objectives. To evaluate and update CVaR in real time, a BNN method based on the t-distribution is employed to accurately update the probability distribution of task completion time. Finally, the SAC-BNN algorithm introduces a perturbation generation network during the training process, enhancing the strategy's perception of high-risk scenarios and adaptability to failure-prone environments. Experimental results demonstrate the superior performance and robustness of SAC-BNN in mitigating extreme delay risks and ensuring reliable task completion. Compared to baseline methods, SAC-BNN consistently achieves lower CVaR and higher on-time task completion rates, reflecting the effectiveness of the ESFMTO strategy in maintaining system stability under uncertain failure conditions. Nevertheless, the strategy does not incorporate more complex network settings, potentially limiting its applicability in highly dynamic environments. In future work, the experimental scale will be further expanded and the performance of ESFMTO will be evaluated in large-scale IIoT scenarios using real industrial platforms. Furthermore, more complex network environments and diverse failure modes should be further investigated to explore more efficient and robust task offloading strategies. ESFMTO will also be considered for application in other IIoT scenarios, such as IoT and vehicular networks, to verify its generalizability and scalability.

CRediT authorship contribution statement

Yubin Yang: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Conceptualization. **Yan Chen:** Conceptualization. **Ningjiang Chen:** Writing – review & editing, Supervision, Funding acquisition, Conceptualization. **Juan Chen:** Writing – review & editing, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (No. 62162003) and the Central Guidance on Local Science and Technology Development Fund of Guangxi Province (No. GuikeZY24212059).

Data availability

Data will be made available on request.

References

[1] P. Zhang, C. Wang, C. Jiang, Z. Han, Deep reinforcement learning assisted federated learning algorithm for data management of IIoT, IEEE Trans. Ind. Inf. 17 (12) (2021) 8475–8484.

[2] M. Sharma, A. Tomar, A. Hazra, Edge computing for industry 5.0: Fundamental, applications and research challenges, IEEE Internet Things J. 11 (11) (2024) 19070–19093.

[3] L. Zhou, L. Zhang, N. Konz, Computer vision techniques in manufacturing, IEEE Trans. Syst. Man Cybern. Syst. 53 (1) (2022) 105–117.

[4] M.I. Khaleel, Failure-aware resource provisioning for hybrid computation offloading in cloud-assisted edge computing using gravity reference approach, Swarm Evol. Comput. 91 (2024) 101704.

[5] T. Long, Y. Ma, Y. Xia, X. Xiao, Q. Peng, J. Zhao, A mobility-aware and fault-tolerant service offloading method in mobile edge computing, in: 2022 IEEE International Conference on Web Services, ICWS, 2022, pp. 67–72.

[6] J. Liang, B. Ma, Z. Feng, J. Huang, Reliability-aware task processing and offloading for data-intensive applications in edge computing, IEEE Trans. Netw. Serv. Manag. 20 (4) (2023) 4668–4680.

[7] B. Zhu, K. Chi, J. Liu, K. Yu, S. Mamtaz, Efficient offloading for minimizing task computation delay of NOMA-based multiaccess edge computing, IEEE Trans. Commun. 70 (5) (2022) 3186–3203.

[8] Z. Bai, Y. Lin, Y. Cao, W. Wang, Delay-aware cooperative task offloading for multi-UAV enabled edge-cloud computing, IEEE Trans. Mob. Comput. 23 (2) (2022) 1034–1049.

[9] T.H. Binh, H. Vo, B.M. Nguyen, H.T.T. Binh, Reinforcement learning for optimizing delay-sensitive task offloading in vehicular edge-cloud computing, IEEE Internet Things J. 11 (2) (2023) 2058–2069.

[10] L. Ai, B. Tan, J. Zhang, R. Wang, J. Wu, Dynamic offloading strategy for delay-sensitive task in mobile-edge computing networks, IEEE Internet Things J. 10 (1) (2022) 526–538.

[11] X. Dai, Z. Xiao, H. Jiang, J.C.S. Lui, UAV-assisted task offloading in vehicular edge computing networks, IEEE Trans. Mob. Comput. 23 (4) (2023) 2520–2534.

[12] T.H. Binh, H.K. Vo, B.M. Nguyen, H.T.T. Binh, S. Yu, Value-based reinforcement learning approaches for task offloading in delay constrained vehicular edge computing, Eng. Appl. Artif. Intell. 113 (2022) 104898.

[13] P. Peng, W. Wu, W. Lin, F. Zhang, Y. Liu, K. Li, Reliable task offloading in sustainable edge computing with imperfect channel state information, IEEE Trans. Netw. Serv. Manag. 21 (6) (2024) 6423–6436.

[14] C. Tang, G. Yan, H. Wu, C. Zhu, Computation offloading and resource allocation in failure-aware vehicular edge computing, IEEE Trans. Consum. Electron. 20 (1) (2023) 1877–1888.

[15] X. Qu, H. Wang, Emergency task offloading strategy based on cloud–edge-end collaboration for smart factories, Comput. Networks. 234 (2023) 109915.

[16] J. Liang, Z. Feng, H. Gao, Y. Chen, J. Huang, H.-L. Truong, Deep reinforcement learning based reliability-aware resource placement and task offloading in edge computing, in: 2024 IEEE International Conference on Web Services, ICWS, pp. 686–695.

[17] F. Zhang, G. Han, L. Liu, M. Martínez-García, Y. Peng, Deep reinforcement learning based cooperative partial task offloading and resource allocation for IIoT applications, IEEE Trans. Netw. Sci. Eng. 10 (5) (2022) 2991–3006.

[18] D. Wei, J. Zhang, M. Shojafar, S. Kumari, N. Xi, J. Ma, Privacy-aware multiagent deep reinforcement learning for task offloading in VANET, IEEE Trans. Intell. Transp. Syst. 24 (11) (2022) 13108–13122.

[19] X. Zhao, M. Liu, M. Li, Task offloading strategy and scheduling optimization for internet of vehicles based on deep reinforcement learning, Ad Hoc Netw. 147 (2023) 103193.

[20] X. Chen, S. Hu, C. Yu, Z. Chen, G. Min, Real-time offloading for dependent and parallel tasks in cloud–edge environments using deep reinforcement learning, IEEE Trans. Parallel Distrib. Syst. 35 (3) (2024) 391–404.

[21] J. Xiong, P. Guo, Y. Wang, X. Meng, J. Zhang, L. Qian, Z. Yu, Multi-agent deep reinforcement learning for task offloading in group distributed manufacturing systems, Eng. Appl. Artif. Intell. 118 (2023) 105710.

[22] T. Pamuklu, A.C. Nguyen, A. Syed, W.S. Kennedy, M. Erol-Kantarci, IoT-aerial base station task offloading with risk-sensitive reinforcement learning for smart agriculture, IEEE Trans. Green Commun. Netw. 7 (1) (2022) 171–182.

[23] D. Xu, X. Su, H. Wang, S. Tarkoma, P. Hui, Towards risk-averse edge computing with deep reinforcement learning, IEEE Trans. Mob. Comput. 23 (6) (2023) 7030–7047.

[24] K.M.U. Ahmed, M. Alvarez, M.H.J. Bollen, Characterizing failure and repair time of servers in a hyper-scale data center, in: 2020 IEEE PES Innovative Smart Grid Technologies Europe. ISGT-Europe, pp. 660–664.

[25] R.T. Rockafellar, S. Uryasev, Conditional value-at-risk for general loss distributions, J. Bank. Financ. 26 (7) (2002) 1443–1471.

[26] X. Jia, H. Gu, Y. Liu, J. Yang, X. Wang, W. Pan, An energy-efficient Bayesian neural network implementation using stochastic computing method, IEEE Trans. Neural Networks Learn. Syst. 35 (9) (2023) 12913–12923.

[27] C. Zhang, J. Bütepage, H. Kjellström, S. Mandt, Advances in variational inference, IEEE Trans. Pattern Anal. Mach. Intell. 41 (8) (2018) 2008–2026.

[28] W. Xu, Y. Yin, N. Chen, H. Tu, Collaborative inference acceleration integrating DNN partitioning and task offloading in mobile edge computing, Int. J. Softw. Eng. Knowl. Eng. 33 (11 & 12) (2023) 1835–1863.

[29] H. Sun, J. Wang, D. Yong, W. Qin, N. Zhang, Deep reinforcement learning-based computation offloading for mobile edge computing in 6G, IEEE Trans. Consum. Electron. 70 (4) (2024) 7428–7493.

[30] D. Zhu, T. Li, H. Tian, Y. Liu, H. Liu, Speed-aware and customized task offloading and resource allocation in mobile edge computing, IEEE Commun. Lett. 25 (8) (2021) 2683–2687.

[31] Y. Li, C. Yang, M. Deng, X. Tang, W. Li, A dynamic resource optimization scheme for MEC task offloading based on policy gradient, in: 2022 IEEE 6th Information Technology and Mechatronics Engineering Conference, ITOEC, pp. 342–345.

[32] H. Li, C. Chen, H. Shan, P. Li, C.C. Chang, S.H. Song, Deep deterministic policy gradient-based algorithm for computation offloading in IoV, IEEE Trans. Intell. Transp. Syst. 25 (3) (2024) 2522–2533.